

18 DE AGOSTO DE 2026

TAREA 3

CONSTRUCCIÓN DE UN FLUJO ANALÍTICO REPRODUCIBLE,
AUTOMATIZADO, AUDITABLE Y MANTENIBLE CON PYTHON

AGILA SOTO ELIZABETH
NIETO PAREDES KATHERINE
MURILLO COELLO KEVIN
GRUPO N°5



	Construcción de un flujo analítico, reproducible, automatizado, auditable y mantenible con Python	Página: 1 of 17 Fecha: 18-08-2026
Taller 3	PYTHON PARA CIENCIA DE DATOS	Realizado por: Grupo N°5

Contenido

Situación	2
Objetivo	2
Flujo Propuesto	2
1. Crear un entorno virtual	3
2. Generación de carpetas	3
3. Carga de Fuentes.....	3
4. Resguardo de Datos crudos (RAW).....	3
5. Lectura controlada	3
6. Control de calidad de datasets	3
7. Estandarización	3
8. Integración	3
9. Diagnóstico de calidad	4
10. Limpieza	4
11. Variable derivada	4
12. Validación final	4
13. Auditoría	4
14. Incorporar Git	4
15. Documentación y Trazabilidad en todo el flujo	4
Bibliografía	17

	Construcción de un flujo analítico, reproducible, automatizado, auditable y mantenible con Python	Página: 2 of 17 Fecha: 18-08-2026
Taller 3	PYTHON PARA CIENCIA DE DATOS	Realizado por: Grupo N°5

Situación

La institución debe construir y actualizar mensualmente un tablero de indicadores provinciales utilizando información proveniente de varias fuentes: registros administrativos internos, CSV de portales públicos, archivos Excel enviados por coordinaciones regionales y JSON generados por una aplicación territorial. El propósito es consolidar esa información para analizar resultados por provincia, mes, indicador y fuente de información

Objetivo

Transformar el proceso actual de consolidación y generación de indicadores en un flujo analítico automatizado, reproducible y trazable, mediante Python y buenas prácticas de gestión de datos, que permita integrar de forma controlada información proveniente de distintas fuentes, validar su calidad, estandarizar su procesamiento y generar resultados consistentes, incorporando documentación, control de versiones y un entorno de ejecución definido, de manera que el proceso pueda ser repetido, auditado y mantenido por cualquier integrante del equipo..

Flujo Propuesto

1. Crear un entorno virtual
2. Generación de carpetas
3. Resguardo de datos crudos
4. Lectura controlada
5. Control de Calidad de Datasets
6. Estandarización
7. Integración
8. Diagnóstico de calidad
9. Limpieza
10. Variable derivada
11. Validación final
12. Auditoría
13. Incorporar GIT
14. Documentación y Trazabilidad durante todo el flujo

	Construcción de un flujo analítico, reproducible, automatizado, auditable y mantenible con Python	Página: 3 of 17 Fecha: 18-08-2026
Taller 3	PYTHON PARA CIENCIA DE DATOS	Realizado por: Grupo N°5

1. Crear un entorno virtual

- Crear entorno de trabajo en Colab

2. Generación de carpetas

- Crear carpetas de trabajo
- Definir rutas de los archivos

3. Carga de Fuentes

- Carga de Dataset de diferentes fuentes.

4. Resguardo de Datos crudos (RAW)

- Guardar una copia original de las fuentes de datos.
- Control de versiones para identificar cada extracción con fecha/hora.
- No modificar ni sobrescribir los archivos originales.

5. Lectura controlada

- Lectura de dataset en formatos JSON, CSV, EXCEL.
- Especificación de formato, separador, codificación y tipos de datos de las variables, normalización de JSON.

6. Control de calidad de datasets

- Existencia de variables requeridas en los dataset.
- Verificación de variable como clave de unión.
- Revisión de valores duplicados y nulos

7. Estandarización

- Homologar nombres de columnas
- Renombrar variables
- Estandarizar códigos de provincias.
- Revisar claves de unión y granularidad.

8. Integración

- Integración progresiva de las diferentes fuentes utilizando la clave de unión.
- Uso de LEFT JOIN para el catálogo de provincias
- Generar variables de control para identificación de registros de cada fuente

	Construcción de un flujo analítico, reproducible, automatizado, auditable y mantenible con Python	Página: 4 of 17 Fecha: 18-08-2026
Taller 3	PYTHON PARA CIENCIA DE DATOS	Realizado por: Grupo N°5

9. Diagnóstico de calidad

- Auditoría de coincidencias.
- Verificación correcta de estandarización en códigos de provincia a 2 dígitos

10. Limpieza

- Eliminar columnas auxiliares generadas durante el merge
- Mantener variables únicas no duplicadas.
- Tratar valores faltantes, NA reemplazar por 0, variables numéricas, SIN INFORMACIÓN variables categóricas.
- Corregir inconsistencias con reglas justificadas.

11. Variable derivada

- Revisión de variables para calculo y metodología
- Cálculo de variables derivadas.

12. Validación final

- Verificar de dataset final (variables completas, cantidad de registros, variable derivada.
- Crear el archivo final limpio y estructurado.
- Dejarlo preparado para el tablero ejecutivo y posteriores modelos predictivos

13. Auditoría

- Reporte automático de Calidad que consiste en mostrar los resultados del procesamiento, generación de código hash de cada archivo con la finalidad de detectar modificaciones posteriores.
- Creación de manifiesto (bitácora de auditoría) para mantener una trazabilidad del procesamiento.

14. Incorporar Git

- Guardar copia del proyecto en Github/Public
- Cargar insumos necesarios. Fuentes de Datos, Informe y Código.

15. Documentación y Trazabilidad en todo el flujo

- Crear un diccionario de datos.

	Construcción de un flujo analítico, reproducible, automatizado, auditable y mantenible con Python	Página: 5 of 17 Fecha: 18-08-2026
Taller 3	PYTHON PARA CIENCIA DE DATOS	Realizado por: Grupo N°5

- Registrar las reglas de limpieza y transformación utilizadas.
- Separar datos crudos, intermedios, procesados y analíticos

A continuación, paso a paso la documentación del proceso:

- Creación del entorno virtual Colab



- Importación de librerías

```
import os
import sys
import hashlib
import json
from datetime import datetime
from pathlib import Path
import pandas as pd
import numpy as np
```

- **Definición de carpetas de trabajo para archivos crudos y procesados**
 - Creación de carpetas de trabajo donde se guardarán los archivos
 - Definición de la ruta de lectura del archivo json
 - Definición de fecha de ejecución de extracción de datos crudos
 - Extracción y resguardo de datos crudos con fecha de ejecución en la carpeta raw
 - Se implementa un mecanismo de control y trazabilidad de las bases de datos originales, manteniendo las fuentes sin modificaciones generando automáticamente una copia versionada antes de iniciar el procesamiento. Cada versión se identifica mediante fecha y hora y se registra su código hash

	Construcción de un flujo analítico, reproducible, automatizado, auditable y mantenible con Python	Página: 6 of 17 Fecha: 18-08-2026
Taller 3	PYTHON PARA CIENCIA DE DATOS	Realizado por: Grupo N°5

```

from datetime import datetime
from shutil import copy2

base_tarea = Path("tarea_educacion")

raw_path = base_tarea / "data" / "raw"
raw_versions_path = base_tarea / "data" / "raw_versions"
proc_path = base_tarea / "data" / "processed"
outputs_path = base_tarea / "data" / "outputs"

raw_path.mkdir(parents=True, exist_ok=True)
proc_path.mkdir(parents=True, exist_ok=True)
outputs_path.mkdir(parents=True, exist_ok=True)

# Crear una versión de las bases originales para esta ejecución
fecha_version = datetime.now().strftime("%Y%m%d_%H%M%S")
version_path = raw_versions_path / fecha_version
version_path.mkdir(parents=True, exist_ok=True)

# Copiar las bases originales sin modificarlas
archivos_originales = [
    "empleados.xlsx",
    "socioeconomico.csv",
    "educacion.json"
]

for archivo in archivos_originales:
    origen = raw_path / archivo
    destino = version_path / archivo

    if origen.exists():
        copy2(origen, destino)
        print(f"Copia creada: {destino}")
    else:
        print(f"ADVERTENCIA: no se encontró {origen}")

```

	Construcción de un flujo analítico, reproducible, automatizado, auditable y mantenible con Python	Página: 7 of 17 Fecha: 18-08-2026
Taller 3	PYTHON PARA CIENCIA DE DATOS	Realizado por: Grupo N°5

- **Lectura controlada**

Esta etapa permite cargar las diferentes fuentes de información de manera controlada, especificando el formato, separador, codificación y tipo de datos de las variables. Esto facilita una correcta interpretación de la información y prepara las bases para su posterior validación, transformación e integración

```
df_empleados = pd.read_excel(
    raw_path / "empleados.xlsx",
    dtype={"cod_prov": str}
)

df_socioeco = pd.read_csv(
    raw_path / "socioeconomico.csv",
    sep=";",
    dtype={"cod_prov": str},
    encoding="utf-8",
)

with open(raw_path / "educacion.json", "r", encoding="utf-8") as f:
    datos_educacion = json.load(f)

df_educacion = pd.json_normalize(datos_educacion)

print(df_empleados )
```

- **Control de Calidad del Dataset.**

Esta etapa permite verificar que cada fuente cumpla con los requisitos mínimos de calidad antes de su integración. Se comprueba la existencia de las variables requeridas y que la variable utilizada como clave de unión no presente valores duplicados ni nulos, garantizando así una integración confiable de las bases de datos.

```
print(df_empleados.columns)
print(df_socioeco.columns)
print(df_educacion.columns)

Index(['cod_prov', 'Provincia', 'total_empleados'], dtype='object')
Index(['cod_prov', 'Provincia', 'monto_prest', 'tipo_cuot'], dtype='object')
Index(['Cod_Prov', 'Provincia', 'Zonal', 'Total_Docentes',
      'Total_Estudiantes'],
      dtype='object')
```


	Construcción de un flujo analítico, reproducible, automatizado, auditable y mantenible con Python	Página: 8 of 17 Fecha: 18-08-2026
Taller 3	PYTHON PARA CIENCIA DE DATOS	Realizado por: Grupo N°5

```
def validar_informacion(df: pd.DataFrame, nombre_fuente: str, columnas_requeridas: set, col_clave: str):
    errores = []

    faltantes = columnas_requeridas - set(df.columns)
    if faltantes:
        errores.append(f"Faltan columnas requeridas: {faltantes}")

    if col_clave in df.columns:
        if df[col_clave].duplicated().any():
            dups = df[col_clave][df[col_clave].duplicated()].tolist()
            errores.append(f"Claves duplicadas detectadas en {col_clave}: {dups}")
        if df[col_clave].isnull().any():
            errores.append(f"Existen valores nulos en la clave {col_clave}")

    if errores:
        raise ValueError(f" FALLÓ CONTRATO DE DATOS [{nombre_fuente}]: {'; '.join(errores)}")

    print(f" Contrato validado con éxito para [{nombre_fuente}] ({len(df)} registros)")
    validar_informacion(df_socioeco, "socioeconomico CSV", {"cod_prov", "monto_prest", "tipo_cuot", "cod_prov"})
    validar_informacion(df_empleados, "empleados Excel", {"cod_prov", "total_empleados", "cod_prov"})
    validar_informacion(df_educacion, "educación JSON", {"Cod_Prov", "Zonal", "Total_Docentes",
                                                         "Total_Estudiantes", "Cod_Prov"})
```

```
Contrato validado con éxito para [socioeconomico CSV] (22 registros)
Contrato validado con éxito para [empleados Excel] (25 registros)
Contrato validado con éxito para [educación JSON] (25 registros)
```

• INTEGRACIÓN DE DATASET

En esta etapa se homogeneizan las variables clave de las diferentes fuentes para garantizar que puedan ser integradas correctamente. Primero, se unifica el nombre de la variable de identificación territorial como cod_provincia. Posteriormente, se estandariza su formato a dos dígitos, de acuerdo con la codificación de la División Político-Administrativa (DPA), mediante el uso de zfill(2). Esto permite que los códigos provinciales mantengan una estructura uniforme y sean utilizados como clave común para la integración de las bases.

Se va a renombrar las variables que se usara como ID código provincia.

```
df_empleados = df_empleados.rename(columns={"cod_prov": "cod_provincia"})
df_socioeco = df_socioeco.rename(columns={"cod_prov": "cod_provincia"})
df_educacion = df_educacion.rename(columns={"Cod_Prov": "cod_provincia"})
df_educacion = df_educacion.rename(columns={"Cod_Prov": "cod_provincia"})
```

Se estandariza la variable cod_provincia a dos digitos como se establece en la DPA

```
df_empleados["cod_provincia"] = df_empleados["cod_provincia"].astype(str).str.zfill(2)
df_socioeco["cod_provincia"] = df_socioeco["cod_provincia"].astype(str).str.zfill(2)
df_educacion["cod_provincia"] = df_educacion["cod_provincia"].astype(str).str.zfill(2)
```

	Construcción de un flujo analítico, reproducible, automatizado, auditable y mantenible con Python	Página: 9 of 17 Fecha: 18-08-2026
Taller 3	PYTHON PARA CIENCIA DE DATOS	Realizado por: Grupo N°5

Se realiza la integración progresiva de las diferentes fuentes utilizando cod_provincia como clave común de unión. Se emplea un left join tomando como base el catálogo de provincias, con el propósito de conservar todas las provincias registradas en el catálogo e incorporarla información disponible de las bases de empleados, socioeconómica y educación.

Asimismo, mediante indicator se generan variables de control (_merge_empleo, _merge_prestamo y _merge_educacion) que permiten identificar posteriormente si los registros de cada fuente fueron correctamente vinculados al consolidado.

```
merge_1 = catalogo_provincias.merge(
    df_empleados,
    on="cod_provincia",
    how="left",
    indicator="_merge_empleo"
)

merge_2 = merge_1.merge(
    df_socioeco,
    on="cod_provincia",
    how="left",
    indicator="_merge_prestamo"
```

```
)

consolidado = merge_2.merge(
    df_educacion,
    on="cod_provincia",
    how="left",
    indicator="_merge_educacion"
)
```

Esta etapa permite verificar el resultado de las uniones realizadas entre las bases de datos. Mediante las variables de control generadas con indicator, se identifica cuántos registros del catálogo de provincias encontraron correspondencia en cada fuente y cuáles no presentaron coincidencias. Esto permite detectar posibles inconsistencias o ausencia de información antes de continuar con el análisis.

	Construcción de un flujo analítico, reproducible, automatizado, auditable y mantenible con Python	Página: 10 of 17 Fecha: 18-08-2026
Taller 3	PYTHON PARA CIENCIA DE DATOS	Realizado por: Grupo N°5

```
print("AUDITORÍA DE COINCIDENCIAS")
print("\n")
print("Presencia en empleo:")
print(consolidado["_merge_empleo"].value_counts())
print("\n")
print("Presencia en socioeconomico:")
print(consolidado["_merge_prestamo"].value_counts())
print("Presencia en educacion:")
print(consolidado["_merge_educacion"].value_counts())
```

AUDITORÍA DE COINCIDENCIAS

```
Presencia en empleo:
_merge_empleo
both          25
left_only      0
right_only     0
Name: count, dtype: int64
```

```
Presencia en socioeconomico:
_merge_prestamo
both          22
left_only      3
right_only     0
Name: count, dtype: int64
Presencia en educacion:
_merge_educacion
both          25
left_only      0
right_only     0
Name: count, dtype: int64
```

▪ Limpieza

Una vez finalizada la auditoría de coincidencias, se eliminan las columnas auxiliares de control generadas durante los merge. Estas variables fueron utilizadas únicamente para verificar la correcta integración de las fuentes, por lo que no son necesarias en la base final

```
columnas_indicadoras = [c for c in consolidado.columns if c.startswith("_merge_")]
consolidado_limpio = consolidado.drop(columns=columnas_indicadoras)
```

 UIDE Powered by Arizona State University	Construcción de un flujo analítico, reproducible, automatizado, auditable y mantenible con Python	Página: 11 of 17 Fecha: 18-08-2026
Taller 3	PYTHON PARA CIENCIA DE DATOS	Realizado por: Grupo N°5

Solo mantenemos la variable de provincia de la base de catalogo

`display(consolidado_limpio)`

`consolidado_limpio = consolidado_limpio.drop(columns=["Provincia_x", "Provincia_y", "Provincia"], errors="ignore")`

	<u>cod_provincia</u>	<u>provincia</u>	<u>Provincia_x</u>	<u>total_empleados</u>	<u>Provincia_y</u>	<u>monto_prest</u>	<u>tipo_cuot</u>	<u>Provincia</u>	<u>Zonal</u>	<u>Total</u>
0	01	Azuay	AZUAY	302003	Azuay	2291410,99	Cuota fija	AZUAY	Zona 6	
1	02	Bolivar	BOLIVAR	67836	Bolivar	654425	Cuota fija	BOLIVAR	Zona 5	
2	03	Cañar	CAÑAR	83165	Cañar	843836,71	Cuota fija	CAÑAR	Zona 6	
3	04	Carchi	CARCHI	65371	Carchi	743820	Cuota fija	CARCHI	Zona 1	
4	05	Cotopaxi	COTOPAXI	165099	Cotopaxi	1304926	Cuota variable	COTOPAXI	Zona 3	
5	06	Chimborazo	CHIMBORAZO	189952	Chimborazo	930375	Cuota fija	CHIMBORAZO	Zona 3	
6	07	El Oro	EL ORO	240214	El Oro	1664224	Cuota fija	EL ORO	Zona 7	
7	08	Esmeraldas	ESMERALDAS	166796	Esmeraldas	401700	Cuota fija	ESMERALDAS	Zona 1	
8	09	Guayas	GUAYAS	1385597	Guayas	1253599,6	Cuota fija	GUAYAS	Zona 5	
9	10	Imbabura	IMBABURA	161452	Imbabura	755558,99	Cuota fija	IMBABURA	Zona 1	
10	11	Loja	LOJA	165981	Loja	714787	Cuota fija	LOJA	Zona 7	
11	12	Los Ríos	LOS RIOS	274997	Los Rios	372973,46	Cuota fija	LOS RIOS	Zona 5	
12	13	Manabí	MANABI	462691	NaN	NaN	NaN	MANABI	Zona 4	
13	14	Morona Santiago	MORONA SANTIAGO	53494	NaN	NaN	NaN	MORONA SANTIAGO	Zona 6	
14	15	Napo	NAPO	39082	Napo	879519,68	Cuota fija	NAPO	Zona 2	
15	16	Pastaza	PASTAZA	31588	Pastaza	979485,63	Cuota fija	PASTAZA	Zona 3	

`display(consolidado_limpio)`

 UIDE Powered by Arizona State University	Construcción de un flujo analítico, reproducible, automatizado, auditable y mantenible con Python	Página: 12 of 17 Fecha: 18-08-2026
Taller 3	PYTHON PARA CIENCIA DE DATOS	Realizado por: Grupo N°5

	cod_provincia	provincia	total_empleados	monto_prest	tipo_cuot	Zonal	Total_Docentes	Total_Estudiantes
0	01	Azuay	302003	2291410,99	Cuota fija	Zona 6	11130	189698
1	02	Bolivar	67836	654425	Cuota fija	Zona 5	2896	46893
2	03	Cañar	83165	843836,71	Cuota fija	Zona 6	3222	57407
3	04	Carchi	65371	743820	Cuota fija	Zona 1	2384	38444
4	05	Cotopaxi	165099	1304926	Cuota variable	Zona 3	6259	111834
5	06	Chimborazo	189952	930375	Cuota fija	Zona 3	7149	108868
6	07	El Oro	240214	1664224	Cuota fija	Zona 7	8876	166937
7	08	Esmeraldas	166796	401700	Cuota fija	Zona 1	8228	157657
8	09	Guayas	1385597	1253599,6	Cuota fija	Zona 5	46185	1038829
9	10	Imbabura	161452	755558,99	Cuota fija	Zona 1	6499	115171
10	11	Loja	165981	714787	Cuota fija	Zona 7	7914	112356
11	12	Los Ríos	274997	372973,46	Cuota fija	Zona 5	9648	220955
12	13	Manabí	462691	NaN	NaN	Zona 4	20748	394439
13	14	Morona Santiago	53494	NaN	NaN	Zona 6	3457	64450
14	15	Napo	39082	879519,68	Cuota fija	Zona 2	2771	41058
15	16	Pastaza	31588	979485,63	Cuota fija	Zona 3	2077	35012
16	17	Pichincha	1182925	3528729,44	Cuota fija	Zona 2	38490	682355

Verificamos en que variables existe NA

```
consolidado_limpio.isnull().sum()
```

```

cod_provincia    0
provincia        0
total_empleados  0
monto_prest      3
tipo_cuot        3
Zonal            0
Total_Docentes   0
Total_Estudiantes 0

```

```
dtype: int64
```

En esta etapa se identifican y tratan los valores faltantes de las variables seleccionadas. Para monto_pres los valores NaN se reemplazan por 0, y las de tipo categórica tipo_cuot con "sin información", con el propósito de evitar valores ausentes en el consolidado final

 UIDE Powered by Arizona State University	Construcción de un flujo analítico, reproducible, automatizado, auditable y mantenible con Python	Página: 13 of 17 Fecha: 18-08-2026
Taller 3	PYTHON PARA CIENCIA DE DATOS	Realizado por: Grupo N°5

```
consolidado_limpio["monto_prest"] = consolidado_limpio["monto_prest"].fillna(0)
```

```
consolidado_limpio["tipo_cuot"] = consolidado_limpio["tipo_cuot"].fillna("Sin información")
display(consolidado_limpio)
```

	cod_provincia	provincia	total_empleados	monto_prest	tipo_cuot	Zonal	Total_Docentes	Total_Estudiantes	
0	01	Azuay	302003	2291410,99	Cuota fija	Zona 6	11130	189698	
1	02	Bolívar	67836	654425	Cuota fija	Zona 5	2896	46893	
2	03	Cañar	83165	843836,71	Cuota fija	Zona 6	3222	57407	
3	04	Carchi	65371	743820	Cuota fija	Zona 1	2384	38444	
4	05	Cotopaxi	165099	1304926	Cuota variable	Zona 3	6259	111834	
5	06	Chimborazo	189952	930375	Cuota fija	Zona 3	7149	108868	
6	07	El Oro	240214	1664224	Cuota fija	Zona 7	8876	166937	
7	08	Esmeraldas	166796	401700	Cuota fija	Zona 1	8228	157657	
8	09	Guayas	1385597	1253599,6	Cuota fija	Zona 5	46185	1038829	
9	10	Imbabura	161452	755558,99	Cuota fija	Zona 1	6499	115171	
10	11	Loja	165981	714787	Cuota fija	Zona 7	7914	112356	
11	12	Los Rios	274997	372973,46	Cuota fija	Zona 5	9648	220955	
12	13	Manabí	462691	0	Sin información	Zona 4	20748	394439	
13	14	Morona Santiago	53494	0	Sin información	Zona 6	3457	64450	
14	15	Napo	39082	879519,68	Cuota fija	Zona 2	2771	41058	
15	16	Pastaza	31588	979485,63	Cuota fija	Zona 3	2077	35012	
16	17	Pichincha	1182925	3528729,44	Cuota fija	Zona 2	38490	682355	
17	18	Tungurahua	236305	1547880	Cuota fija	Zona 3	6830	128435	

 UIDE Powered by Arizona State University	Construcción de un flujo analítico, reproducible, automatizado, auditable y mantenible con Python	Página: 14 of 17 Fecha: 18-08-2026
Taller 3	PYTHON PARA CIENCIA DE DATOS	Realizado por: Grupo N°5

■ Variable Derivada

Se calcula la tasa de docentes por cada 100 estudiantes, relacionando el número total de docentes con el número total de estudiantes. Se utiliza np.where para evitar divisiones por cero en los casos en que no existan estudiantes registrados.

```
consolidado_limpio["tasa_docente_estu"] = np.where(
    consolidado_limpio["Total_Estudiantes"] > 0,
    (consolidado_limpio["Total_Docentes"] / consolidado_limpio["Total_Estudiantes"]) * 100,
    0.0
)
print("Tasa de docentes por estudiante:")
print(consolidado_limpio["tasa_docente_estu"])
```

Tasa de docentes por estudiante:

```
0    5.867221
1    6.175762
2    5.612556
3    6.201228
4    5.596688
5    6.566668
6    5.316976
7    5.218925
8    4.445871
9    5.642914
10   7.043683
```

```
display(consolidado_limpio)
```

	cod_provincia	provincia	total_empleados	monto_prest	tipo_cuot	Zonal	Total_Docentes	Total_Estudiantes	tasa_docente_
0	01	Azuay	302003	2291410,99	Cuota fija	Zona 6	11130	189698	5.86
1	02	Bolívar	67836	654425	Cuota fija	Zona 5	2896	46893	6.17
2	03	Cañar	83165	843836,71	Cuota fija	Zona 6	3222	57407	5.61
3	04	Carchi	65371	743820	Cuota fija	Zona 1	2384	38444	6.20
4	05	Cotopaxi	165099	1304926	Cuota variable	Zona 3	6259	111834	5.59
5	06	Chimborazo	189952	930375	Cuota fija	Zona 3	7149	108868	6.56
6	07	El Oro	240214	1664224	Cuota fija	Zona 7	8876	166937	5.31
7	08	Esmeraldas	166796	401700	Cuota fija	Zona 1	8228	157657	5.21
8	09	Guayas	1385597	1253599,6	Cuota fija	Zona 5	46185	1038829	4.44
9	10	Imbabura	161452	755558,99	Cuota fija	Zona 1	6499	115171	5.64
10	11	Loja	165981	714787	Cuota fija	Zona 7	7914	112356	7.04
11	12	Los Ríos	274997	372973,46	Cuota fija	Zona 5	9648	220955	4.36
12	13	Manabí	462691	0	Sin información	Zona 4	20748	394439	5.26
13	14	Morona Santiago	53494	0	Sin información	Zona 6	3457	64450	5.36

■ Auditoria

En esta etapa se guardan los resultados del procesamiento en formatos CSV y Excel, organizados en las carpetas correspondientes de datos procesados y productos finales. Posteriormente, se calcula

	Construcción de un flujo analítico, reproducible, automatizado, auditable y mantenible con Python	Página: 15 of 17 Fecha: 18-08-2026
Taller 3	PYTHON PARA CIENCIA DE DATOS	Realizado por: Grupo N°5

el hash SHA-256 de cada archivo generado, lo que permite verificar su integridad y detectar posibles modificaciones posteriores a su generación.

Además, la función `calcular_sha256()` garantiza que cada archivo tenga una huella digital única que puede utilizarse para fines de trazabilidad y control de los resultados.

```
from pathlib import Path

processed_path = Path("/content/drive/MyDrive/taller_educacion/data/processed")
outputs_path = Path("/content/drive/MyDrive/taller_educacion/data/outputs")

processed_path.mkdir(parents=True, exist_ok=True)
outputs_path.mkdir(parents=True, exist_ok=True)

ruta_salida_csv = processed_path / "consolidado_provincial_informacion_202601.csv"
ruta_salida_excel = outputs_path / "tablero_indicadores_informacion_202601.xlsx"

consolidado_limpio.to_csv(
    ruta_salida_csv,
    index=False,
    encoding="utf-8"
)

consolidado_limpio.to_excel(
    ruta_salida_excel,
    index=False
)

def calcular_sha256(ruta_archivo: Path) -> str:
    hasher = hashlib.sha256()
    with open(ruta_archivo, "rb") as f:
        hasher.update(f.read())
    return hasher.hexdigest()

hash_csv = calcular_sha256(ruta_salida_csv)
hash_excel = calcular_sha256(ruta_salida_excel)
```

El manifiesto permite mantener una trazabilidad básica del procesamiento y de los archivos generados, facilitando su identificación y posterior verificación.

 UIDE Powered by Arizona State University	Construcción de un flujo analítico, reproducible, automatizado, auditable y mantenible con Python	Página: 16 of 17 Fecha: 18-08-2026
Taller 3	PYTHON PARA CIENCIA DE DATOS	Realizado por: Grupo N°5

```
import json

manifiesto = {
    "archivo_procesado": ruta_salida_csv.name,
    "archivo_excel": ruta_salida_excel.name,
    "filas_totales": len(consolidado_limpio),
    "columnas_totales": len(consolidado_limpio.columns),
    "sha256_csv": hash_csv,
    "sha256_excel": hash_excel
}

ruta_manifiesto = outputs_path / "manifiesto_ejecucion.json"

with open(ruta_manifiesto, "w", encoding="utf-8") as f:
    json.dump(
        manifiesto,
        f,
        ensure_ascii=False,
        indent=4
    )

print(f"Manifiesto guardado en: {ruta_manifiesto}")
```

Manifiesto guardado en: tarea_educacion/data/outputs/manifiesto_ejecucion.json

```
print(" Proceso finalizado con éxito.")
print(f" - CSV SHA256: {hash_csv}")
```

▪ Diccionario de Datos

Nombre del Campo (Encabezado Columna)	Descripción del Campo	
<u>cod_prov</u>	codigo provincia	
<u>provincia</u>	Provincia	
<u>zonal</u>	Numero de Zonal	
<u>total_docentes</u>	numero de docentes	
<u>total_estudiantes</u>	numero de estudiantes matriculados	
<u>prestamo</u>	prestamo total por provincia	
<u>tipo_cuota</u>	tipo de cuota que tiene	
<u>total_empleados</u>	número de personas con empleo	

	Construcción de un flujo analítico, reproducible, automatizado, auditable y mantenible con Python	Página: 17 of 17 Fecha: 18-08-2026
Taller 3	PYTHON PARA CIENCIA DE DATOS	Realizado por: Grupo N°5

Bibliografía

<https://pandas.pydata.org/docs/>